Cast-History Metric Checking Guide

Version: C6f/C6f3
Purpose: Guide students who manually inspect cast-history outputs for individual C projects and verify whether the reported fields, metrics, and classifications make sense.

1. Goal of Manual Checking

The goal is not merely to check whether the tool runs. The goal is to determine whether the output tells a semantically reasonable story about how pointer values are cast, stored, passed, returned, and used.

In particular, we want to answer:

```
For each surface-wild pointer, especially void *, is it actually wild?
Or is it structured by concrete type use, data fields, callbacks, guards,
return aliases, or cross-translation-unit summaries?
```

Students should check whether the tool correctly identifies:

```
1. What entity is being analyzed.
2. What cast history it has.
3. What pointer operations are performed on it.
4. How those operations are mediated.
5. Whether the final behavior classification is reasonable.
6. Whether JSON and text output agree at the stable classification layer.
```

2. Files to Inspect

For each analyzed project, the main output directory should look like:

```
test-output/real-projects/<project-name>/
  commands.log
  *.txt
  summaries/
    *.cast-summary.json
  project-summary-index.json
  project-summary-index-used-for-final-import.json
  import-round-01-index-input.json
  import-round-02-index-input.json
```

Students should primarily inspect:

```
*.txt
```

for human-readable explanations, and:

```
summaries/*.cast-summary.json
```

for machine-readable stable results.

The `.txt` file is better for understanding the analysis. The JSON file is better for checking stable metrics and exporting tables.

3. Recommended Checking Order

For each project, follow this order.

Step 1: Confirm the project ran correctly

Open:

```
commands.log
```

Check for:

```
1. The expected source files were analyzed.
2. The expected number of import rounds completed.
3. There were no tool crashes.
4. The final summaries were generated.
```

If a source file was skipped, check whether it appears in `compile_commands.json` and whether it was excluded by `SOURCE_FILTER_REGEX` or `SOURCE_EXCLUDE_REGEX`.

Step 2: Open the final text output for one source file

For each `.txt` output, first scan these sections:

```
Function summaries
Direct cast histories
Storage-location cast histories
Pointer use summaries
Semantic pointer-operation summaries
Pointer behavior metrics
```

```
Function parameter behavior metrics
Function behavior classifications
Unsupported / approximated language features
```

Do not start with the final classification only. First understand the low-level evidence.

Step 3: Identify important entities

Focus on entities that contain:

```
void *
char *
unsigned char *
function pointers
field paths such as box.payload or ctx.box.payload
return aliases
pointer arithmetic
deref-readwrite operations
external or unresolved callees
```

These are most relevant to the paper's "not all void* is wild" argument.

Examples of important entities:

```
run::x
run::box.payload
run::ctx.box.payload
handle_int::x
ho::pvho
demo::pv
demo::pi
```

4. How to Read Cast Histories

A simple cast history:

```
run::x : void * => int *
```

means the value represented by `run::x` is viewed as `int *`.

A branch cast:

```
run::x : void * => ⟨[tag] int *, [!(tag)] char *⟩
```

means the pointer is interpreted differently under different branch guards.

A callback-wrapped history:

```
run::x : void * => handle_int⌜int *⌝handle_int
```

means the concrete interpretation happens inside the callback target `handle_int`.

A higher-order symbolic history:

```
ho::pvho : void * => char * => void * => $0.$0
```

means the value is locally viewed as `char *`, converted back to `void *`, and then passed into the callback parameter `$0`.

A data-field-mediated history:

```
run::x : void * => int *
run::box.payload : void * => int *
```

together with pointer-use evidence showing:

```
origin=data-field
field=run::box.payload
```

means the concrete use reaches `x` through the field `box.payload`.

5. How to Read Pointer Use Summaries

Pointer use summaries explain what operations are performed on pointers.

Common operation kinds:

```
init-rhs
stored
call-arg
returned
deref-read
```

```
deref-write
deref-readwrite
pointer-arithmetic
```

Important examples:

```
deref-readwrite as=int *
```

means the pointer is dereferenced and written through as an `int *`.

```
stored as=void *
```

means the pointer was stored, often into a field or variable.

```
call-arg as=void *
```

means the pointer was passed as an argument.

```
returned-flow as=int *
```

means the pointer participates in a return flow or return-alias continuation.

When checking whether a classification makes sense, always verify that the pointer-use evidence supports it. For example, `ConcreteSpecializedWrite` should normally have a concrete dereference/write operation such as:

```
typed-deref-readwrite as=int *
```

or:

```
concreteWrite=yes
```

6. Stable Classification Fields

The most important C6f fields are:

```
stableClassification
stableParamClassification
```

```
semanticMediation
paperWildness
```

These fields are the main outputs students should compare across projects.

A typical entity-level stable line:

```
stableClassification:
  entity=run::x,
  behavior=ConcreteSpecializedWrite,
  mediation=DataField,
  family=DataFieldMediatedConcreteSpecialization,
  risk=Low
```

A typical parameter-level stable line:

```
stableParamClassification:
  param=run.$0,
  behavior=ConcreteSpecializedWrite,
  mediation=Callback,
  family=CallbackMediatedConcreteSpecialization,
  risk=Low
```

7. Meaning of `behavior`

The `behavior` field describes the observed pointer behavior.

Common values:

```
NotWildLike
ConcreteSpecializedWrite
ConcreteSpecializedRead
ByteView
ByteViewWrite
CallbackDependentByteViewPointer
BranchTypeSplitNoDeref
ExternalUnknown
OpaqueTransport
```

Interpretation:

```
NotWildLike
```

The entity does not behave like a wild pointer in the observed evidence.

```
ConcreteSpecializedWrite
```

A surface-wild pointer, usually `void *`, is used concretely and written through, such as `int *`.

```
ByteView / ByteViewWrite
```

The pointer is viewed as `char *` or byte-like memory. This may represent byte-level access, serialization, copying, or low-level reinterpretation.

```
CallbackDependentByteViewPointer
```

The behavior depends on a callback or higher-order function parameter.

```
BranchTypeSplitNoDeref
```

The type interpretation splits by branch, but there may be no concrete dereference evidence at that point.

```
OpaqueTransport
```

The pointer is transported or forwarded without enough evidence of concrete use.

```
ExternalUnknown
```

The behavior flows into an unresolved or external boundary.

8. Meaning of `semanticMediation.kind`

The `semanticMediation.kind` field explains how the behavior reaches the entity.

Expected values include:

```
Direct
DataField
NestedDataField
```

```
Callback
GuardedCallback
DataFieldCallback
NestedDataFieldCallback
ReturnAlias
ExternalBoundary
Mixed
Unknown
```

Direct

The entity is used directly.

Example:

```
void run(void *x) {
  int *p = x;
  *p += 1;
}
```

Expected:

```
mediation=Direct
family=DirectConcreteSpecialization
```

DataField

The pointer flows through a data field.

Example:

```
box.payload = x;
int *p = box.payload;
*p += 1;
```

Expected:

```
mediation=DataField
fields=run::box.payload
family=DataFieldMediatedConcreteSpecialization
```

NestedDataField

The pointer flows through a nested field path.

Example:

```
ctx.box.payload = x;
int *p = ctx.box.payload;
*p += 1;
```

Expected:

```
mediation=NestedDataField
fields=run::ctx.box.payload
family=NestedDataFieldMediatedConcreteSpecialization
```

Callback

The pointer is interpreted through a known callback target.

Example:

```
void (*cb)(void *) = handle_int;
cb(x);
```

Expected:

```
mediation=Callback
targets=handle_int
edges=function-pointer-call
family=CallbackMediatedConcreteSpecialization
```

GuardedCallback

The callback target depends on a branch condition.

Example:

```
if (tag)
  cb = handle_int;
else
```

```
  cb = handle_char;

cb(x);
```

Expected:

```
mediation=GuardedCallback
family=GuardedCallbackMediatedSpecialization
```

The cast history should preserve guarded alternatives:

```
[tag] int *
[!(tag)] char *
```

ReturnAlias

The behavior is continued through a return value that aliases a parameter.

Example:

```
void *idv(void *x) {
  int *p = x;
  return p;
}
```

Expected evidence:

```
returned-flow
return-alias-continuation
returnAlias=yes
```

ExternalBoundary

The value flows into an unresolved external callee or unknown boundary.

Example:

```
extern void sink(void *);
sink(x);
```

Expected:

```
mediation=ExternalBoundary
externalBoundary=yes
edges=external-direct-call
```

Do not classify a local recursive function or higher-order callback parameter as `ExternalBoundary`.

9. Meaning of `semanticMediation.semanticFamily`

The `semanticFamily` field is the paper-facing behavior family.

Common values:

```
DirectConcreteSpecialization
DataFieldMediatedConcreteSpecialization
NestedDataFieldMediatedConcreteSpecialization
CallbackMediatedConcreteSpecialization
GuardedCallbackMediatedSpecialization
CallbackDependent
ReturnAliasMediatedConcreteSpecialization
ExternalUnknownBoundary
ByteView
OpaqueTransport
NotWildLike
MixedMediatedConcreteSpecialization
```

Students should check whether this field matches the evidence.

For example:

```
fields=run::box.payload
mediation=DataField
```

should not normally have:

```
semanticFamily=DirectConcreteSpecialization
```

unless the output is intentionally reporting a direct behavior for a different entity.

10. Meaning of `paperWildness`

The `paperWildness` field is intended for paper-level interpretation.

Important subfields:

```
surfaceKind
semanticFamily
semanticRiskTier
interpretation
```

Example:

```
paperWildness:
  surfaceKind=void-interface
  semanticFamily=ConcreteSpecialization
  semanticRiskTier=Low
  interpretation=surface-wild interface with stable concrete specialization
```

This means the code looks wild at the surface because it uses `void *`, but the observed behavior is actually structured and concrete.

Students should check whether `paperWildness` is too pessimistic or too optimistic compared with the evidence.

11. Risk Tiers

Common risk levels:

```
Low
Moderate
High
FunctionParameterDependent
Unknown
```

General interpretation:

```
Low
```

Stable concrete behavior with limited ambiguity.

```
Moderate
```

Mixed structure, byte-view behavior, branch-dependent behavior, callback dependence, or structurally more complex history.

```
High
```

Potentially unsafe or highly ambiguous reinterpretation, such as multiple incompatible concrete writes, pointer arithmetic, or unresolved unsafe flows.

```
FunctionParameterDependent
```

The behavior depends on a function parameter, usually a callback parameter.

Students should not judge risk only from `void *`. A `void *` can be low risk if it consistently specializes to one concrete type.

12. Key Invariants to Check

Students should check these invariants.

Invariant 1: Concrete write requires concrete dereference evidence

If:

```
behavior=ConcreteSpecializedWrite
```

then there should usually be evidence like:

```
typed-deref-readwrite as=int *
concreteWrite=yes
writeDerefTypes: int *
```

Invariant 2: DataField mediation should identify a field when possible

If:

```
mediation=DataField
```

then the output should ideally include:

```
fields=...
```

Examples:

```
fields=run::box.payload
fields=run::ctx.box.payload
```

Some older or composite cases may not yet populate `fields`; mark those as metadata-completeness issues, not necessarily analysis failures.

Invariant 3: NestedDataField should use a nested path

If:

```
mediation=NestedDataField
```

then the field should look nested:

```
run::ctx.box.payload
```

rather than:

```
run::box.payload
```

Invariant 4: Callback mediation should have callback evidence

If:

```
mediation=Callback
```

then at least one of the following should appear:

```
function-pointer-call
field-function-pointer-call
higher-order-call
higher-order-callback
callback=yes
targets=...
callbacks=...
```

Invariant 5: GuardedCallback should preserve guards

If:

```
mediation=GuardedCallback
```

then there should be branch evidence such as:

```
[tag] int *
[!(tag)] char *
```

or a guarded target set.

Invariant 6: ExternalBoundary should only mean truly unresolved/external

If:

```
mediation=ExternalBoundary
```

then:

```
externalBoundary=yes
```

should also appear.

The edge:

```
external-direct-call
```

should not appear when:

```
externalBoundary=no
```

Local recursive calls, known function pointers, and higher-order callback parameters should not be labeled as external boundaries.

Invariant 7: Function summaries should reflect parameter behavior

If an entity-level parameter has a propagated history:

```
run::x : void * => int *
```

then the function summary should usually reflect:

```
run($0 : void * => int *)
```

For callback cases:

```
run($0 : void * => handle_int⌜int *⌝handle_int)
```

For guarded callback cases:

```
run($0 : void * => ⟨[tag] ..., [!(tag)] ...⟩)
```

If pointer-use evidence exists but the cast history or function summary is missing, flag it as a propagation asymmetry.

Invariant 8: Text and JSON stable fields should agree

The `.txt` output and the corresponding JSON summary should agree on:

```
behavior
mediation
family
risk
fields
targets
edges
flags
```

When in doubt, treat JSON as the stable source and text as explanatory output.

13. Common Patterns and Expected Classifications

Direct concrete specialization

Code:

```
void run(void *x) {
  int *p = x;
  *p += 1;
}
```

Expected:

```
behavior=ConcreteSpecializedWrite
mediation=Direct
family=DirectConcreteSpecialization
risk=Low
```

Data-field-mediated specialization

Code:

```
box.payload = x;
int *p = box.payload;
*p += 1;
```

Expected:

```
behavior=ConcreteSpecializedWrite
mediation=DataField
family=DataFieldMediatedConcreteSpecialization
fields=box.payload
```

Nested data-field specialization

Code:

```
ctx.box.payload = x;
int *p = ctx.box.payload;
*p += 1;
```

Expected:

```
mediation=NestedDataField
fields=ctx.box.payload
family=NestedDataFieldMediatedConcreteSpecialization
```

Callback-mediated specialization

Code:

```
void (*cb)(void *) = handle_int;
cb(x);
```

Expected:

```
mediation=Callback
targets=handle_int
family=CallbackMediatedConcreteSpecialization
```

Guarded callback-mediated specialization

Code:

```
if (tag)
  cb = handle_int;
else
  cb = handle_char;

cb(x);
```

Expected:

```
mediation=GuardedCallback
family=GuardedCallbackMediatedSpecialization
```

Expected cast history:

```
[tag] int *
[!(tag)] char *
```

Higher-order callback-dependent behavior

Code:

```
void ho(void *(*pf)(void *), void *x) {
  pf(x);
}
```

Expected inside `ho`:

```
mediation=Callback
family=CallbackDependent
edges=higher-order-call
externalBoundary=no
```

It should not be reported as:

```
external-direct-call
ExternalUnknownBoundary
```

unless the target is genuinely unresolved at the relevant level.

Recursive direct call

Code:

```c
void f(void *x, int n) {
  if (n <= 0) return;
  int *p = x;
  f(p, n - 1);
}
```

Expected:

```
mediation=Direct
externalBoundary=no
```

A local recursive call should not be classified as an external boundary.

External unknown boundary

Code:

```c
extern void sink(void *);
void run(void *x) {
  sink(x);
}
```

Expected:

```
mediation=ExternalBoundary
family=ExternalUnknownBoundary
externalBoundary=yes
edges=external-direct-call
```

14. What to Flag During Manual Review

Students should flag the following cases.

A. Missing cast history but present pointer-use evidence

Example problem:

```
Pointer use summaries show run::x receives deref-readwrite as int *
but Direct cast histories do not show run::x : void * => int *
and function summary remains run($0 : void *)
```

This suggests propagation reached pointer operations but not cast histories.

B. Incorrect external boundary

Flag if:

```
external-direct-call
```

appears for:

```
local recursive call
known same-TU function
known imported function
function pointer with known target
higher-order callback parameter
```

unless there is a real unresolved external call.

C. Lost branch guard

Flag if code has:

```
if (tag)
  cb = handle_int;
```

```
else
  cb = handle_char;
```

but output collapses to:

```
⟨| int *, char * |⟩
```

without guards, or worse, only one target.

Expected:

```
[tag] int *
[!(tag)] char *
```

D. Wrong mediation kind

Examples:

```
Data-field code reported as Direct
Callback code reported as Direct
Higher-order callback reported as ExternalBoundary
Nested field path reported as simple DataField when nested path is visible
```

E. Missing field path

If the behavior clearly flows through:

```
ctx.box.payload
```

but the output only reports:

```
ctx.payload
```

or omits the field entirely, flag it.

F. Duplicate or suspiciously repeated evidence

Some duplicates are harmless, but flag cases where repeated evidence changes classification or inflates counts.

G. Unsupported feature appears important

If the source uses features such as:

```
container_of
complex macros
casts through integers
arrays of structs
unions
pointer arithmetic into struct fields
heap object aliasing
```

and the output looks incomplete, note that the case may require unsupported or approximate language modeling.

15. Validation Commands

After running a project, run:

```
python3 scripts/validate_classification_labels.py
  --root test-output/real-projects/<project-name>
```

Then run:

```
python3 scripts/validate_cast_summary_schema.py
  --root test-output/real-projects/<project-name>
```

For stricter checking:

```
python3 scripts/validate_classification_labels.py
  --strict-vocabulary
  --root test-output/real-projects/<project-name>
```

Strict-mode failures may indicate metadata-completeness issues rather than semantic failures. Students should record them, but should not automatically treat every strict-mode issue as an analysis bug.

16. Exporting Rows for Review

Use:

```
python3 scripts/extract_stable_classifications.py
  --root test-output/real-projects/<project-name>
  --out results/<project-name>-stable-classifications.csv
```

Students can use the CSV to inspect:

```
subject
behavior
mediation
family
risk
fields
targets
edges
flags
```

This CSV is also the basis for paper tables.

17. Manual Review Template

For each reviewed project, students should fill out a short report.

```
Project:
Reviewer:
Date:
Tool commit:
Project commit:
Command used:
Validation result:

1. Overall status
   - Did the project run successfully?
   - Number of source files analyzed:
   - Number of JSON summaries generated:
   - Any crashes or skipped files?

2. Main observed semantic families
   - DirectConcreteSpecialization:
   - DataFieldMediatedConcreteSpecialization:
   - NestedDataFieldMediatedConcreteSpecialization:
   - CallbackMediatedConcreteSpecialization:
   - GuardedCallbackMediatedSpecialization:
   - CallbackDependent:
   - ByteView:
```

```
   - OpaqueTransport:
   - ExternalUnknownBoundary:
   - Other:

3. Examples that look correct
   Example 1:
     Source location:
     Entity:
     Expected behavior:
     Observed behavior:
     Why it looks correct:

   Example 2:
     Source location:
     Entity:
     Expected behavior:
     Observed behavior:
     Why it looks correct:

4. Suspicious or incorrect cases
   Case 1:
     Source location:
     Entity:
     Observed output:
     Why it may be wrong:
     Suspected cause:
       [ ] missing cast-history propagation
       [ ] missing pointer-use propagation
       [ ] lost branch guard
       [ ] wrong mediation label
       [ ] external-boundary false positive
       [ ] unsupported language feature
       [ ] other

5. Notes on unsupported or approximated language features

6. Suggested follow-up tests
```

18. Reviewer Guidelines

When reviewing, prefer evidence-based judgments.

Do not say:

```
This looks wrong because the label seems strange.
```

Instead say:

```
This looks wrong because the code calls a known local callback target,
but semanticMediation reports ExternalBoundary and edge=external-direct-call.
```

or:

```
This looks wrong because pointer-use summaries show a data-field dereference
through run::ctx.box.payload, but stableClassification reports mediation=Direct.
```

Always connect the suspected issue to source code and output evidence.

19. Summary of What “Good Output” Looks Like

Good output should satisfy these principles:

```
1. Surface void* is not automatically treated as semantically wild.
2. Concrete dereference/write evidence is reflected in behavior.
3. Data-field mediation records field paths.
4. Nested fields preserve full paths.
5. Callback mediation identifies targets or callback dependence.
6. Branch-dependent targets preserve guards.
7. ExternalBoundary is reserved for genuinely unresolved boundaries.
8. Return-alias evidence remains visible.
9. Function summaries agree with entity-level histories.
10. Stable JSON fields match the text explanation.
```

The final goal is to support reliable paper-level claims about how C programs use casts and `void *`: many cases that look wild at the surface are actually structured by types, fields, callbacks, guards, and summaries.